Preliminaries

Azotobacter vinelandii is a gram negative, nitrogen fixing bacteria and the NCBI Genebank record under CP092752.1 is analyzed here.

Reading FASTA file with python.

```
def read_fasta(fasta_file):
    sequences = []
    headers = []
    with open(fasta_file, 'r') as f:
        sequence = ''
        for line in f:
            if line.startswith('>'):
                headers.append(line.strip())
                if sequence:
                    sequences.append(sequence)
                    sequence = ''
            else:
                sequence += line.strip()
        if sequence:
            sequences.append(sequence)
    return headers, sequences
```

```
Header: >CP092752.1 Azotobacter vinelandii strain AEIV chromosome, complete genome
Sequence (first 50 bases): GTGTCCGTGGAACTTTGGCAGCAGTGCGTCGAGCTGCTGCGCGACGAGCT
```

Reading GTF file

```
import gffpandas.gffpandas as gffpd

# Load the GTF file
gtf_file = "CP092752_1.gtf"
gff_df = gffpd.read_gff3(gtf_file)
# Convert GTF data into a dataframe
gtf_df = gff_df.df

# Display the first few rows of the dataframe
print(gtf_df.head())
```

```
       seq_id            source         type  start   end score strand phase  \
0  CP092752.1           Genbank         gene      1  1437     .      +     .
1  CP092752.1  Protein Homology          CDS      1  1434     .      +     0
2  CP092752.1  Protein Homology  start_codon      1     3     .      +     0
3  CP092752.1  Protein Homology   stop_codon   1435  1437     .      +     0
4  CP092752.1           Genbank         gene   1466  2569     .      +     .

                                        attributes
0  gene_id "AVAEIV_000001"; transcript_id ""; gbk...
1  gene_id "AVAEIV_000001"; transcript_id "unassi...
2  gene_id "AVAEIV_000001"; transcript_id "unassi...
3  gene_id "AVAEIV_000001"; transcript_id "unassi...
4  gene_id "AVAEIV_000002"; transcript_id ""; gbk...
```

Obtaining 100 bases upstream and 3 bases downstream of all genes in the sequence using start column of the gtf file. (Here starting position of type gene is considered and extracted 100 bp upstream and 3 bp downstream for promoter search)

```
sense_strand_genes = gtf_df[(gtf_df['strand'] == '+') & (gtf_df['type'] == 'gene')]
genes_sequences=[]
genes_sequences_w = []
for index, row in sense_strand_genes.iterrows():
    gene_start = int(row['start'])
    gene_end = int(row['end'])

    # Extract the sequence of the gene
    gene_seq = sequence[gene_start - 100:gene_start+2]
    genes_sequences.append(gene_seq)
```

10 Sequences were printed out to verify consistency with the factor bacteria genes start with Methionine (ATG) and verify last 3 of the selected sequence (3bp ds) is ATG.

```
for i in range(1,11):
    print(f"seq{i}:", genes_sequences[i])
```

```
seq1: CCGAGCTGCGCGAAACCGACGCGGATATCCGCGAGGACTACAAGAACCTGCTGCGTACCCTGACCACCTGAGCCAGACGCAACCGATTGGGGACTGACCATG
seq2: TCCGACTCCAACAGCAGCGCCTTGCTCCAGGAAGCCGATAATGACGATTCCGCCTATGTCGTCATGCCGATGCGGCTGTAAACCTGCAACCCCGCCTCGATG
seq3: TGAAGCATTGCGTGAAGGCTGGCGCACGGATACGCCAGTTGCCCTGTTCCACGTGGAACAGGGCCGTATCACCCAGACCCACGATCGGGAGTGAAGGTAATG
seq4: CACCGACCAGATCTTCAACACCCTGATGGGCGATGCCGTCGAACCGCGTCGCGAATTCATCGAGAACAACGCGTTGGCGGTGGCGAATCTGGATGTGTGATG
seq5: GAAGTCGCATGTGGCCCTTTGCGACGAGGATTTTCGATTCTGTAACGGCGCCGTTTCTTGCCTAGAATGATCGCTTGTCGACCACACGGAGTCCACGACATG
seq6: CGTAGTCTGGCTCACAAAAATCACCCTGTCCTGAGGCTGCGACGTAAAAGGCTGCGAGTATACCCGATTCCCGGCCCCCTCCACTCTGTGCAAACGCCCATG
seq7: GACCGCCGGAACCCGCTGTCGCTTTTCCGGGCGGAAGGAAGGAAGCCATACGACCGTGCTATCTTGCAGGACCCGGTCCGAGCGCCGATGGAGCGCGCCATG
seq8: TGAGCAGCATTGAAAAAACGGCTTTACGCCGCGATCCGGAGAGAAATCCGGCGGCGCATGACGTTTCATGCGAAGGCACATCATAAAGGGATTCGGCGCATG
seq9: ACTGGCGGTGATGGACAAGGTGCGCAACCCGTAACCTCCGCGTTTTCCCTCCGTCGAGGGAACCGCCTTTCGCTTCCATGGATCAAGGACGATCGCTCCATG
seq10: GCCCGCGAAGGCGGACGCTGGCAGGGATGCGCCGGCTGACGGGCTGCGCCGCCGCCGACGCCTTGGGTACACTGCGCCGAATTTTTCCGGAGGCGGGCGATG
```

Q1. Local Search for Promoter

Smith-Waterman Algorithm for Local Intact Alignment was used.
Scoring matrix was obtained using the recursive relation

$$F(i,j) = \max \begin{pmatrix} F(i-1,j-1) \pm m & \text{Match or mismatch} \\ F(i-1,j) - g & \text{Gap in } x \\ F(i,j-1) - g & \text{Gap in } y \\ 0 & \end{pmatrix}$$

```
def scorematrix_local(seq_x, seq_y, score_m, score_mm, gap_penalty):
    len_x = len(seq_x)
    len_y = len(seq_y)
    basic_mat = np.zeros((len_x + 1, len_y + 1))

    # Recurrence relation
    for k_x in range(1, len_x + 1):
        for k_y in range(1, len_y + 1):
            score_match = basic_mat[k_x - 1, k_y - 1] + prom_base(seq_x[k_x - 1], seq_y[k_y - 1], score_m, score_mm)
            score_gap_y = basic_mat[k_x, k_y - 1] + gap_penalty(1)
            score_gap_x = basic_mat[k_x - 1, k_y] + gap_penalty(1)
            basic_mat[k_x, k_y] = max(score_match, score_gap_x, score_gap_y, 0)
    return basic_mat
```

If the max score of the score matrix is 4 or above (atleas 4 consecative W) it is considered as a hit and the percentage of sequences with probable promoter region is calculated.

```
def search_promoter_region(genes, promoter_seq, score_m=1, score_mm=-1):
    promoter_hits = 0
    positions = []
    positions_h = []

    for gene in genes:
        positions_i = []
        # Perform traceback to find alignment
        align_x, align_y,score_matrix = traceback_local(gene, promoter_seq, score_m, score_mm, gap_penalty)
        max_score = np.max(score_matrix)
        if max_score > 3:
            start_pos = gene.find(align_x.replace('-', ''))
        else:
            start_pos = -1
        if start_pos != -1:  # If a valid alignment is found
            promoter_hits += 1
            positions_h.append(start_pos)
            positions_i.append(start_pos)
        positions.append(positions_i)

    promoter_percentage = (promoter_hits / len(genes)) * 100
    return promoter_percentage, positions, positions_h
```

It has been found that 72.58% of the selected genes have a possible promoter according to the results.

```
promoter_seq = "WWWW"
promoter_percentage, positions, positions_h = search_promoter_region(genes_sequences_w, promoter_seq)
print(f"Percentage of genes with potential promoters: {promoter_percentage}%")
```

```
Percentage of genes with potential promoters: 72.58464079273328%
```

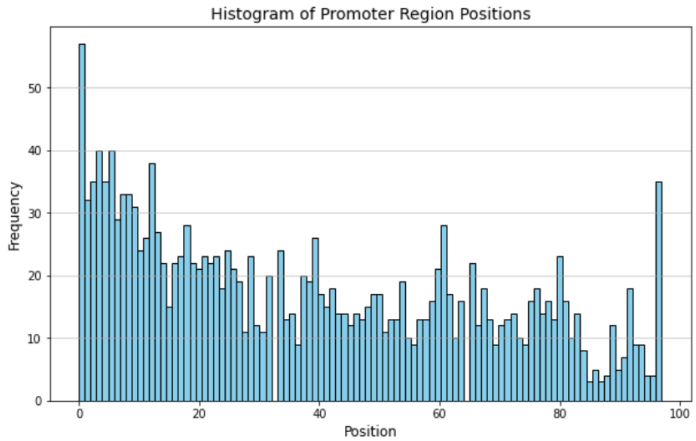
Upstream positions were trace-backed using an algorithm similar to manually tracing back (taken from the lab class scilab code)

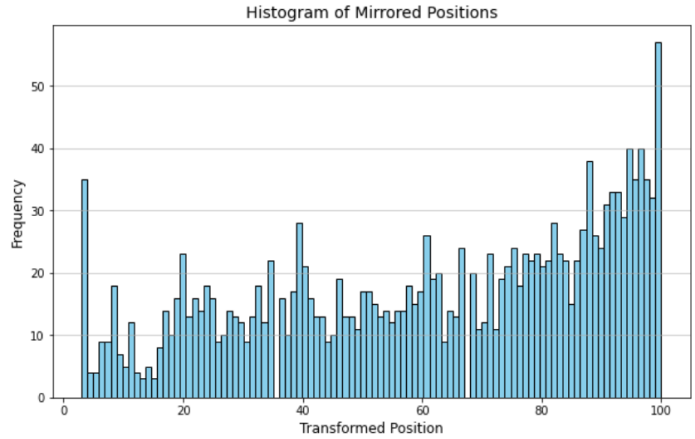
```
while k_x > 0 and k_y > 0:
    current_score = score_matrix[k_x, k_y]
    diag_score = score_matrix[k_x - 1, k_y - 1]
    gap_x_score = score_matrix[k_x, k_y - 1]
    gap_y_score = score_matrix[k_x - 1, k_y]

    if current_score == diag_score + prom_base(seq_x[k_x - 1], seq_y[k_y - 1], score_m, score_mm):
        align_x.append(seq_x[k_x - 1])
        align_y.append(seq_y[k_y - 1])
        k_x -= 1
        k_y -= 1
    elif current_score == gap_x_score + gap_penalty(1):
        align_x.append('-')
        align_y.append(seq_y[k_y - 1])
        k_y -= 1
    elif current_score == gap_y_score + gap_penalty(1):
        align_x.append(seq_x[k_x - 1])
        align_y.append('-')
        k_x -= 1
    else:
        break
```

Creating Histogram for the position of the upstream position

```
# Create the histogram
plt.figure(figsize=(10, 6))
plt.hist(positions_h, bins=100,color='skyblue', edgecolor='black')

# Add titles and labels
plt.title('Histogram of Promoter Region Positions', fontsize=14)
plt.xlabel('Position', fontsize=12)
plt.ylabel('Frequency', fontsize=12)

# Show grid
plt.grid(axis='y', alpha=0.75)

# Show the plot
plt.show()
```

Two position histograms were created due to the ambiguity in position naming convention Considering positions starting from the furthest away from the gene as zero and going in sense direction and Considering positions starting from the starting of the gene as zero and going in anti- sense direction (simply the mirror of the original graph).

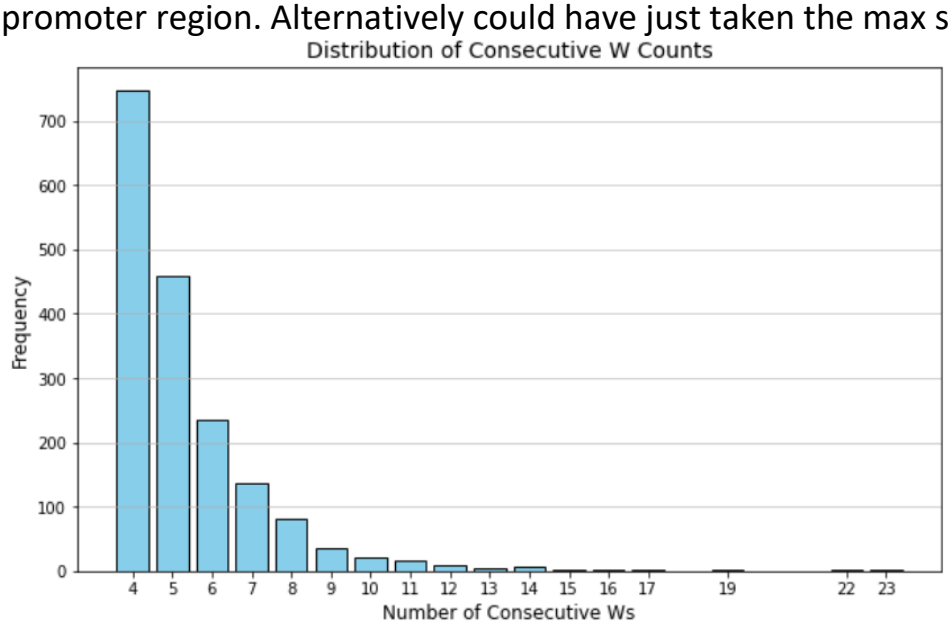




Q2. Distribution of number of consecutive Ws

```
def count_consecutive_Ws(sequence):
    max_count = 0
    current_count = 0
    for base in sequence:
        if base == 'W':
            current_count += 1
            if current_count > max_count:
                max_count = current_count
        else:
            current_count = 0
    return max_count
```

A simple consecutive count algorithm was executed only for the sequences with probable promoter region. Alternatively could have just taken the max score of the score matrix



Q3. Calculating Position Probability Matrix

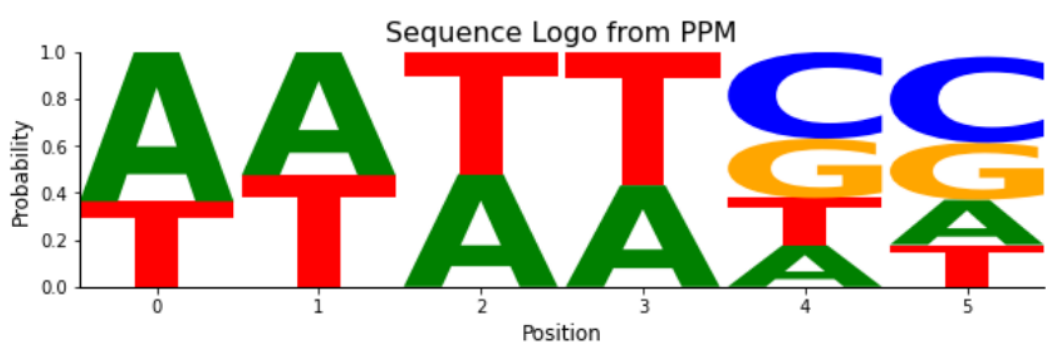
```
# Initialize the position probability matrix
position_prob_matrix = np.zeros((6, 4))  # 6 positions, 4 bases (A, C, G, T)
# Fill the position probability matrix
for seq in df_sequences['sequence']:
    for i, base in enumerate(seq):
        if base == 'A':
            position_prob_matrix[i, 0] += 1
        elif base == 'C':
            position_prob_matrix[i, 1] += 1
        elif base == 'G':
            position_prob_matrix[i, 2] += 1
        elif base == 'T':
            position_prob_matrix[i, 3] += 1
# Convert counts to probabilities
num_sequences = len(extracted_sequences)
if num_sequences > 0:
    position_prob_matrix = position_prob_matrix / num_sequences
```

Firstly, counted each base pair at the respective position and converted to a probability by dividing by total number of sequences considered.

	A	C	G	T
Position 1	0.636519	0.000000	0.000000	0.363481
Position 2	0.525028	0.000000	0.000000	0.474972
Position 3	0.480091	0.000000	0.000000	0.519909
Position 4	0.431741	0.000000	0.000000	0.568259
Position 5	0.176337	0.369170	0.253129	0.201365
Position 6	0.191695	0.366325	0.243458	0.178612

PPM was converted to a web-logo using python logomaker package

```
position_prob_matrix = position_prob_matrix
ppm_df = pd.DataFrame(position_prob_matrix, columns=['A', 'C', 'G', 'T'])
# Create a WebLogo
logo = logomaker.Logo(ppm_df, shade_below=.5, fade_below=.5, color_scheme='classic')
```



Q4. Statistical Alignment

Above Position probability matrix was used for statistical alignment, for a particular subsequence of length 6 alignment score is calculated as the natural logarithm of multiplication of probability of the given base at respective position. To implement the algorithm the property of log(a*b)=log(a)+log(b) has been used. Also note that in numpy np.log refers to natural logarithm but not the log base 10. A small constant has been addded to the probability to avoide getting infinity for log(0).

```
>>> import numpy as np
>>> np.log([1, np.e, np.e**2, 0])
array([  0.,   1.,   2., -inf])
```

```
def get_score(seq, ppm):
    score = 0
    for i, base in enumerate(seq):
        if base == 'A':
            score += np.log(ppm[i, 0] + 1e-5)
        elif base == 'C':
            score += np.log(ppm[i, 1] + 1e-5)
        elif base == 'G':
            score += np.log(ppm[i, 2] + 1e-5)
        elif base == 'T':
            score += np.log(ppm[i, 3] + 1e-5)
    return score
```

Position for the subsequence with highest score is scanned and the process is iterated over every sequence that was extracted 100bp upstream and 2bp downstream of the start of the genes given in gtf file.

```
def get_motif_score(seq, ppm):
    n = len(seq)
    m = ppm.shape[0]
    max_score = float('-inf')
    max_pos = -1
    max_seq = ""
    # Iterate through all possible positions
    for i in range(n - m + 1):
        sub_seq = seq[i:i + m]
        score = get_score(sub_seq, ppm)
        if score > max_score:
            max_score = score
            max_pos = i
            max_seq = sub_seq
    return max_score, max_pos, max_seq
```

```
def scan_promoters(genes_sequences, ppm, score_thresh):
    results = []
    for gene_seq in genes_sequences:
        if len(gene_seq) < ppm.shape[0]:
            continue
        sm_score, sm_pos, sm_seq = get_motif_score(gene_seq, ppm)
        if sm_score > score_thresh:
            results.append((sm_score, sm_pos, sm_seq))
    return results
```

Final resulted are as below to threshold as a hit a sub sequence that has more match score than “TTAAGG” has been selected as the threshold it is best to remind that in statistical alignment match percentage is heavily depend on this threshold and it is arbitral defined.

```
------------------------------
 Match Percentage : 70.27250206440958 %
------------------------------
```

When score of “TTTCCC” was considered for the minimum alignment percentage was

```
------------------------------
 Match Percentage : 97.02725020644097 %
------------------------------
 Entropy (w):
  [[  0.93455692 -11.51292546 -11.51292546   0.37427362]
  [  0.74199628 -11.51292546 -11.51292546   0.64179927]
  [  0.65251998 -11.51292546 -11.51292546   0.73219766]
  [  0.54636985 -11.51292546 -11.51292546   0.82112146]
  [ -0.34905124   0.38980177   0.01244644  -0.21632839]
  [ -0.26554175   0.38206783  -0.02650425  -0.33623073]]

 Information Content (su):
  [[ 0.59486302 -0.          -0.          0.13604143]
  [ 0.38956915 -0.         -0.          0.3048364 ]
  [ 0.31326898 -0.         -0.          0.38067615]
  [ 0.23589006 -0.         -0.          0.46660998]
  [-0.06155056  0.14390293  0.00315055 -0.04356101]
  [-0.05090305  0.13996114 -0.00645269 -0.06005486]]

 Net Information Content per Position:
  [0.73090446 0.69440555 0.69394513 0.70250003 0.04194191 0.02255054]

 Total Information Content:
  2.886247612902961
------------------------------
  Match Percentage : 70.27250206440958 %
------------------------------
```

More over information content of each position of the ppm has been calculated using,

$$I_j = \frac{1}{\ln(2)} \sum_N p_{j,N} \ln\left(\frac{p_{j,N}}{p_{0,N}}\right)$$

```python
def ppm_info(ppm, p0):
    ppm = np.array(ppm)
    p_mat = np.ones_like(ppm) / p0
    w = np.log(ppm * p_mat + 1e-5)
    su = (ppm * w )/np.log(2)
    return w, su
```

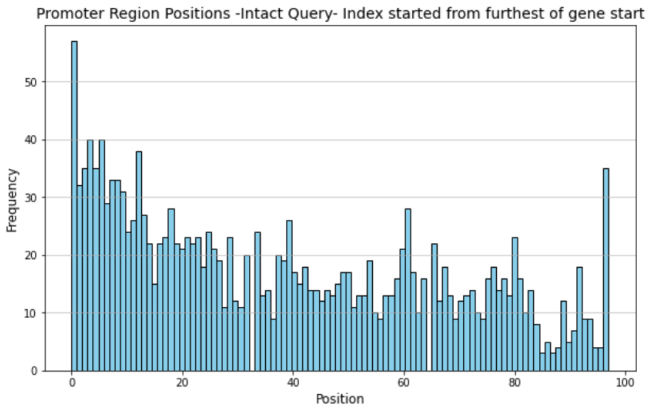
Aligned position with maximum score for the alignments above threshold was printed.

```
Promoter Scanning Results:
Score: -4.4958, Position: 24, Matched Sub-Sequence: ATATCC
Score: -5.2306, Position: 78, Matched Sub-Sequence: TAAACC
Score: -4.9635, Position: 53, Matched Sub-Sequence: AATTCA
Score: -5.0222, Position: 29, Matched Sub-Sequence: ATTTTC
Score: -5.0432, Position: 17, Matched Sub-Sequence: AAATCA
Score: -4.9764, Position: 22, Matched Sub-Sequence: TTTTCC
Score: -4.3956, Position: 43, Matched Sub-Sequence: AAATCC
Score: -4.9764, Position: 42, Matched Sub-Sequence: TTTTCC
Score: -4.9764, Position: 82, Matched Sub-Sequence: TTTTCC
Score: -4.7730, Position: 92, Matched Sub-Sequence: AAATGC
Score: -4.6704, Position: 50, Matched Sub-Sequence: AAAACC
Score: -5.2817, Position: 29, Matched Sub-Sequence: ATATGG
Score: -4.3159, Position: 65, Matched Sub-Sequence: AATTCC
Score: -4.7730, Position: 77, Matched Sub-Sequence: AAATGC
Score: -5.3179, Position: 94, Matched Sub-Sequence: AAAACA
Score: -5.3537, Position: 82, Matched Sub-Sequence: TTTTGC
Score: -5.3850, Position: 54, Matched Sub-Sequence: TTTTCG
```

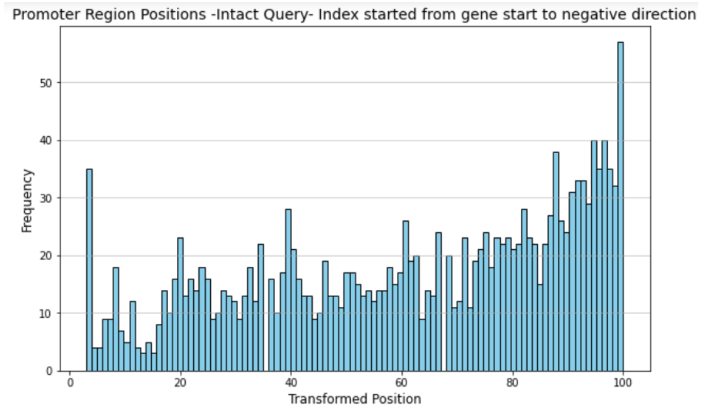
Q5. Comparison

Intact Query

Distribution of the position of the promoter using Smith-Waterman Intact Query.
(L- Positions Indexed Considering positions Starting from the furthest away from the gene, R- Considering positions Starting from the starting of the gene as zero)

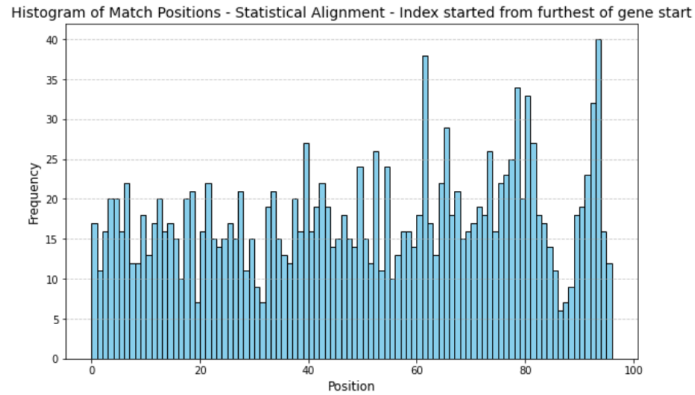



```
Mean: 38.24687144482366
Standard Deviation: 28.591975391096074
```

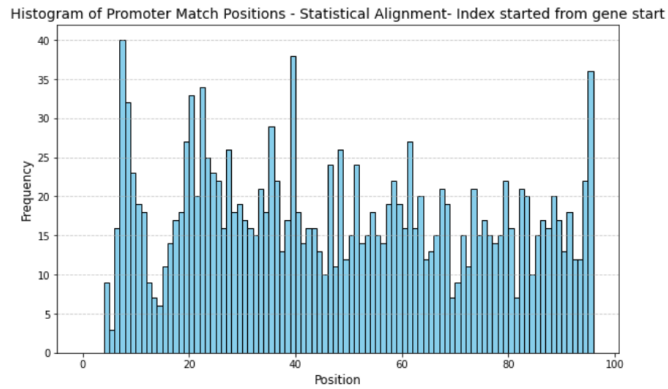



```
Mean: - 61.75312855517634
Standard Deviation: 28.591975391096074
```

Statistical Alignment




```
Mean: 49.98178613396005
Standard Deviation: 27.76181252453743
```




```
Mean: - 50.01821386603995
Standard Deviation: - 27.76181252453743
```

L- Positions Indexed Considering positions Starting from the furthest away from the gene, R- Considering positions Starting from the starting of the gene as zero)

It is observed that Local Intact Query has a position distribution that is skewd but the distribution of the statistical alignement seemed to be more uniform.

While percentage for the local intact query 72.58% was fixed the threshold for the statistical alignment was not exactly given if if minimum match score was taken as of "TTAAGG" 70.27% match percentage would be given and if the minimum score was set to of "TTTCCC" match percentage of "97.03%" was given.
While Local Intact Query gives more exact answers (had to replace A,T s with W as local intact query finds exact matches) it is computationally expensive, on the other side statistical alignment provide a scalable solution as well as being flexible for a wider range of possible promoters.